

30 June 2026

architecture

①

React Frontend → Load balancer will run

Panel will interact with ^{this} to see what is going on inside the load balancer computer.

Tails feature to connect multiple websocket servers to the load balancer. so load balancer acts like a client to the backend servers.

Frontend is running on localhost:5173

First & most important part of Frontend is the useTelemetry Tails() hook (custom).

Its purpose is to manage websocket connections where each connection is represented as a Tail.

Each Tail has its own Backend IP address.

websocket connection

logs

events

Connection status

Prebuilt React Hooks

useState - store values & triggers UI updates.
stores tails & active tails.

useRef - stores values without causing re-renders.
stores websocket object

useEffect - runs side effects (cleanup, API calls, closes sockets when subscription component unmounts)

use Memo() → caches expensive values so they aren't recreated

Creates only one initial tab.

use Callback() → caches functions so they aren't recreated every render.

Prevents unnecessary creation of helper functions.

MAX-LOGS=25;

MAX-EVENTS=30;

→ limits how much data each tab stores.

log 1

log 2

log 25

when 26th log arrives,
1st log disappears.
saves memory

function createID() → To create a unique ID for log, event, tabs.

function createTab() → returns

{id, name, backendHost, connected,

connecting, logs, events}

every time we press add server button, this function is called

const initialTab = useMemo(() => createTab(), []);

useSocketStorage

const socketRef = useRef(new Map());

This stores

(3)

TabID



WebSocket Object

tab1



WebSocket # 1

tab 2



WebSocket # 2

Don't cause re-render to UI

connect() → suggest function

const host = backendHost, trim();

Input:- 192.168.1.51

if (!host) Please enter valid backend
server IP

Close previous socket

existingSocket.close()

Choose protocol

ws: wss:

Create URL:-

ws: localhost:8080/lb-server-channel

backendIP = 192.168.1.51

Frontend always connects to

localhost:8080

The backend receives backendIP

Create socket
new WebSocket(...) as a query parameter.

Then store socket

4

map
↓
tabId
↓
socket

update UI

connecting = true

socket.onOpen

↓
Run after successful connection

connected = true

connecting = false

socket.onmessage

→ Runs whenever backend sends data

```
{ CPU: 42,  
  ram: 60...  
}
```

addTab()

→ Creates a new Tab,

Also switches to the new tab.

In a nutshell, what React Frontend does is it sends backend IP to the webSocket client installed in the load balancer & says please make a connection to that.

Then the socket.onmessage() function is triggered whenever new logs or events are sent

and logs() is printed in the status card.

1 July 2026

Websocket Spring Application (5)

→ Runs on load balancer

at port 8080

Overall Architecture

React Frontend
(localhost : 5173)

↓ websocket

Spring Boot Websocket Proxy
(localhost : 8080)

↓ Java HTTP websocket
Backend Telemetry Server Client
(192.168.x.x : 8086 / telemetry)

We have 3 classes here

① websocket-for-load-balancer-application.java
Entry point of Spring boot application
& it creates beans.

② websocket-config.java
Registers /li-server-channel

③ load-balancer-websocket-handler.java

↓

This is the entire application
It receives websocket connection from React.
It connects to backend servers.
It forwards messages both ways.

after Spring starts, it scans for @Components (6)
& @Configuration, finds it & connects to it.

@Configuration

@EnableWebSocket

WebSocketConfig

registry.addHandler (loadBalancer
WebSocketHandler, "/lb-server-
channel")

This tells Spring that whenever someone connects to

ws://localhost:8080/lb-server-
channel

use LoadBalancer WebSocketHandler

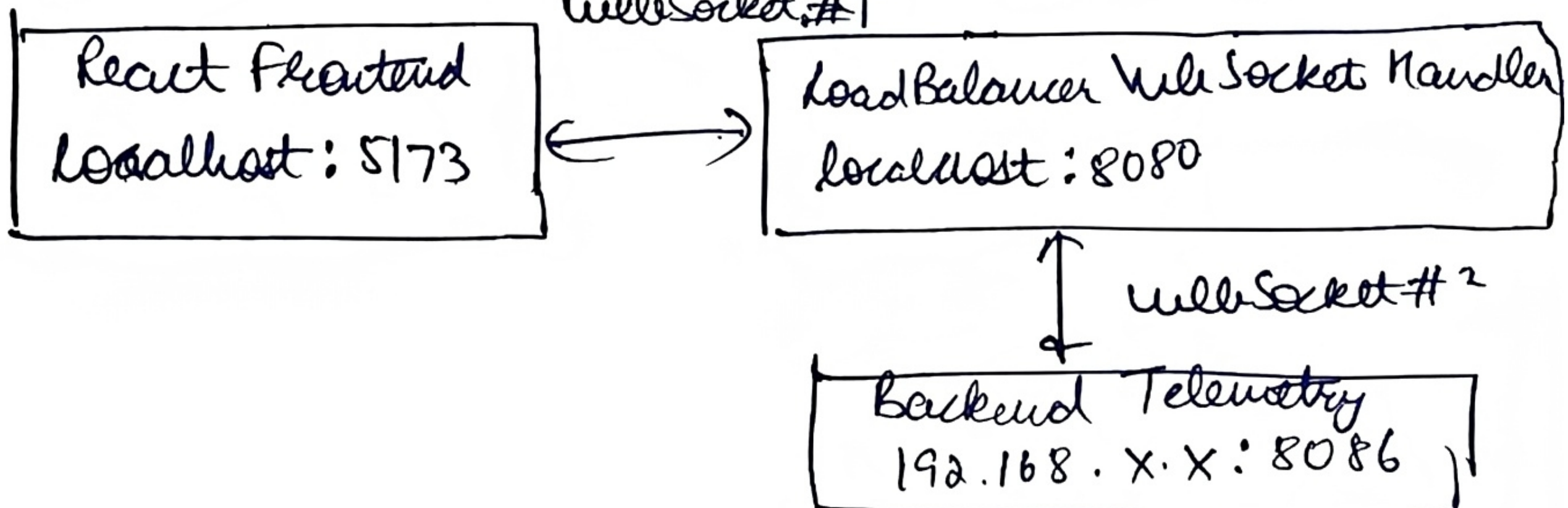
Now the most important class

LoadBalancerWebSocketHandler → Heart
extends TextWebSocketHandler
@Component

default BackendPort = 8086;

// so make sure your Telemetry
collector runs on port 8086

* This class handles two different WebSocket
Connections simultaneously.
WebSocket #1



2 different WebSocket APIs being used. (7)
Spring WebSocket

import org.springframework.web.socket.
WebSocketSession

This represents

React $\longleftrightarrow$ Spring

Java HTTP Client WebSocket

import java.net.http.WebSocket

This represents

Spring $\longleftrightarrow$ Backend

Spring becomes the client here.

Object Mapper

Java Object $\longleftrightarrow$ JSON conversion
jackson library

private final HttpClient httpClient;

java's built HTTP/WebSocket
client

upstream connections

↓

Map<String, WebSockets>

= new concurrent

It stores

HashMap<>();

Frontend session ID

↓

Backend WebSocket

eg:- Session A

↓

Backend server
192.168.1.20

Session B

↓

Backend server
192.168.1.51

after Connection established ()

8

→ This is the first method after
React connects .

React

↓

new WebSocket(...)

↓

Spring

↓

after Connection established ()

Reads backend IP

@Override

public void afterConnectionEstablished
(WebSocketSession session)

{ String backendIp = getQueryParameter

(session, getUri(),

"backendIp")

If there is error in

backendIp

→ sends JSON (error) to

React

otherwise

your connection is established

(This only tells React)